Fundamentos de Programación
PR1 - 20252

Fecha límite de entrega: **01/04/2026 a las 23:59** ⏱

Enunciado

Esta práctica tiene tres partes (algoritmo, programa y proceso). **Las tres son obligatorias.**

1ª parte. Algoritmo.[70%].

Desarrolla un algoritmo que lea los datos de consumo de una red inteligente (Smart Grid) desde un fichero, calcule estadísticas generales como el consumo total y la media, identifique el pico máximo de potencia y clasifique las lecturas según la tarifa aplicada.

Este algoritmo gestionará una secuencia de lecturas de potencia y **generará un resumen detallado del consumo**, identificando en cuál de las tres tarifas disponibles (Valley, Flat o Peak) se ha producido el consumo más elevado.

Entrada de datos:

El algoritmo debe leer los datos de consumo eléctrico desde un fichero. En la primera línea, el fichero contiene un valor entero que indica el **número total de lecturas** que hay que procesar. A continuación, cada línea del fichero contiene la información de una lectura:

- Potencia consumida (en kW). Los valores pueden tener decimales.
- Código de la tarifa aplicada (1 para Valley, 2 para Flat, 3 para Peak).

El usuario deberá introducir por teclado el nombre del fichero de datos según el siguiente mensaje: LOAD DATA FROM FILE. ENTER FILE NAME >>

Suponemos que el usuario introduce el nombre de un fichero existente y con los datos adecuados, así que no será necesario comprobar si la lectura de datos ha sido correcta o no. El sistema puede gestionar hasta un máximo de 100 lecturas.

Junto con este enunciado, se adjuntan los ficheros **grid1.txt** y **grid2.txt** con datos de prueba.

Procesamiento de datos:

El algoritmo debe calcular la siguiente información:

1. Cargar los datos del fichero en vectores de potencia y tarifas, antes de comenzar el procesamiento.
2. Para cada lectura cargada:validar si la potencia es legal (valor superior o igual a 0.0). Si es negativa, se debe contabilizar como una lectura errónea.
3. Calcular el consumo total acumulado y la potencia media de todas las lecturas válidas.
4. Determinar el valor de la potencia máxima encontrada y registrar el código de la tarifa asociada.
5. Contabilizar cuántas lecturas se han realizado para cada tipo de tarifa (Valley, Flat, Peak).

Salida de datos:

El algoritmo debe mostrar la siguiente información:

1. Un resumen de análisis (ANALYSIS SUMMARY) que incluya el consumo total, la potencia media y el pico máximo con su tarifa correspondiente.
2. Un desglose del número de lecturas por cada tipo de tarifa.
3. El recuento de lecturas descartadas por errores en el valor de potencia y los valores erróneos encontrados.

La estructura del algoritmo, así como algunos de los tipos de datos, constantes y variables, ya están declarados y deben ser el punto de partida para la resolución:

Pseudocódigo

```
const
    MAX_READINGS: integer = 100       {Maximum number of readings allowed}
    {...}
end const

type
    tTariffType = {VALLEY, FLAT, PEAK} {Enum to represent tariff types}
end type

algorithm SmartGridAnalysis

    var
        powerData: vector[MAX_READINGS] of {...};  {Array for power values}
        tariffData: vector[MAX_READINGS] of {...}; {Array for tariff types}
        {...}

        totalConsumption: real;       {Sum of valids readings}
        totalElements: integer;       {Number of readings}
        validReadingsCount: integer; {Number of valid readings}
        {...}
```

```
    i, j: integer;
    filename: string;          {Name of the file to read}
    fileToRead: file;          {File to open}
end var

{Data input from file}
writeString("--- Smart Grid Analysis System (PR1) ---");
writeString("LOAD DATA FROM FILE. ENTER FILE NAME >> ");
filename := readString();
fileToRead := openFile(filename);

{Read number of readings from file}
{...}

{Ensure we don't exceed array capacity}
if totalElements > MAX_READINGS then
    totalElements := MAX_READINGS;
end if

{Read power data and tariffs from file}
{...}

closeFile(fileToRead);

{Data Processing}
writeString("Processing records...");

{Initialize counters}
totalConsumption := 0.0;
{...}

for j := 1 to totalElements do
    {Process valid/non-valid readings}
    if {...} then
        {Register non-valid readings}
        {...}
    else
        {Register valid readings and calculate total consumption}
        {...}

        {Check for peak power}
        {...}

        {Update tariff counters}
        {...}

    end if
end for

{Data Output}
writeString("----------------------------------------");
writeString("ANALYSIS SUMMARY");
writeString("----------------------------------------");

if validReadingsCount > 0 then
    {Calculate average power}
    {...}

    {Output stats and distribution}
    writeString("Total consumption: ");
    writeReal(totalConsumption);
    writeString(" kW");
    {...}
    writeString("----------------------------------------");
```

```
    else
        writeString("No valid data found to perform analysis.");
    end if

end algorithm
```

Ejemplos de ejecución

Ejemplo 1

--- Smart Grid Analysis System (PR1) ---
LOAD DATA FROM FILE. ENTER FILE NAME >> grid1.txt

Processing 5 records…

ANALYSIS SUMMARY

Total consumption: 9.05 kW
Average power: 1.81 kW
Maximum peak: 3.10 kW (Tariff: PEAK)

Distribution by Tariff:
- Valley intervals: 2
- Flat intervals: 2
- Peak intervals: 1

Discarded readings: 0

Ejemplo 2

--- Smart Grid Analysis System (PR1) ---
LOAD DATA FROM FILE. ENTER FILE NAME >> grid2.txt

Processing 6 records...

ANALYSIS SUMMARY

Total consumption: 13.10 kW
Average power: 3.28 kW
Maximum peak: 5.60 kW (Tariff: FLAT)

Distribution by Tariff:
- Valley intervals: 1
- Flat intervals: 2
- Peak intervals: 1

Discarded readings: 2
-0.50 kW
-1.20 kW

2a parte. Código y pruebas. [30%].

Codificar en lenguaje C el algoritmo diseñado anteriormente, ejecutarlo y **probarlo** hasta asegurar que funciona correctamente diseñando los casos de prueba necesarios.

Ejecutar y superar los juegos de prueba automáticos disponibles en la herramienta CodeGrade.

CodeGrade se basa en una comprobación literal de la salida obtenida por el programa sometido a prueba, con los resultados esperados de los juegos de prueba. Por ello, los textos de la interfaz de usuario deben ser exactamente idénticos a los esperados. Podéis recuperar dichos textos de los ejemplos de ejecución anteriores. Hay que tener en cuenta que al copiar y pegar textos, se pueden generar caracteres ocultos que hacen que la comparación de los textos literales sea incorrecta a pesar de que los textos literales aparentemente sean idénticos. Si tenéis problemas al pasar los tests, aseguraos que los literales son los correctos.

3a parte. Proceso de resolución.

Explicar el proceso seguido para solucionar la PR1, mediante la grabación de un vídeo corto (máximo 2 minutos, pero menos de un minuto sería ideal) en el que expliquéis y documentéis cómo habéis resuelto esta PR1.

Este vídeo es una prueba de la autoría de vuestro ejercicio, y lo revisaremos solo en caso de que haya dudas al respecto. Por lo tanto, el vídeo debe demostrar que habéis seguido el proceso que os sugerimos: primero diseñar el algoritmo en pseudocódigo, después codificar el programa y probarlo e iterar estos pasos hasta que el resultado sea correcto y responda al problema que plantea el enunciado.

¿Ha habido algo que os ha costado entender o que no teníais claro cómo resolver? ¿Qué pasos habéis hecho para convertir el algoritmo en un código C, con qué dificultades os habéis encontrado, y cómo las habéis resuelto? Y también qué herramientas habéis usado para codificar el algoritmo y probar el programa, qué pruebas habéis hecho, cómo las habéis pensado, qué tipo de errores habéis encontrado al compilar y cómo los habéis resuelto. Y finalmente, qué es lo que os ha costado más, qué unidades de los recursos habéis consultado en cada paso, si habéis consultado al profesorado, en los foros, etc. **No se trata que leáis y expliquéis el código**. Esto ya lo podemos ver en el algoritmo y el programa. Queremos que nos dejéis claro que el ejercicio lo habéis hecho vosotros.

Es imprescindible entregar las tres partes de la práctica para que se corrija.